TRƯỜNG ĐẠI HỌC BÁCH KHOA - ĐẠI HỌC QUỐC GIA TP.HCM



BÀI THỰC HÀNH VI XỬ LÝ - VI ĐIỀU KHIỂN

LAB 2: TIMER INTERRUPT AND LED SCANNING

NGUYỄN ĐỖ KHÁNH TRÌNH - 2353237

30/09/2026

Mục lục

Nội Dung Thực Hành **3**

1 Cấu hình chung **3**
1.1 Timer 2 . . . 3
1.2 Phân bổ chân . . . 3

2 Exercise 1 **4**
2.1 Report 1: Schematic . . . 4
2.2 Report 2: Source code . . . 4
2.3 Short question . . . 6

3 Exercise 2 **6**
3.1 Report 1: Schematic . . . 6
3.2 Report 2: Source code . . . 7
3.3 Short question . . . 7

4 Exercise 3 **7**
4.1 Report 1: Hàm update7SEG . . . 7
4.2 Report 2: Hàm ngắt timer . . . 8

5 Exercise 4 **8**
5.1 Report 1: Hàm ngắt timer . . . 8

6 Exercise 5 **9**
6.1 Report 1: Hàm updateClockBuffer . . . 9

7 Exercise 6 **9**
7.1 Report 1: Bỏ dòng 1 . . . 10
7.2 Report 2: Dòng 1 đổi thành setTimer0(1) . . . 10
7.3 Report 3: Dòng 1 đổi thành setTimer0(10) . . . 10

8 Exercise 7 **11**
8.1 Report 1: Vòng lặp while trong main . . . 11

9 Exercise 8 **11**
9.1 Report 1: Hàm main . . . 11

10 Exercise 9 **13**
10.1 Report 1: Schematic . . . 13
10.2 Report 2: Hàm updateLEDMatrix . . . 13

11 Exercise 10 **15**
11.1 Report 1: Mô tả giải pháp và source code . . . 15

Nội Dung Thực Hành

Code và Schematic có thể được xem trực tiếp và tải về tại GitHub Repository: Microcontroller Reports

1 Cấu hình chung

1.1 Timer 2

Clock hệ thống 8MHz (HSI). Timer 2 được cấu hình Prescaler = 7999, Counter Period = 9 và bật ngắt TIM2 global interrupt:

$$f_{timer} = \frac{8\,\text{MHz}}{7999+1} = 1000\,\text{Hz}, \qquad f_{interrupt} = \frac{1000\,\text{Hz}}{9+1} = 100\,\text{Hz} \;\Rightarrow\; T = 10\,\text{ms}$$

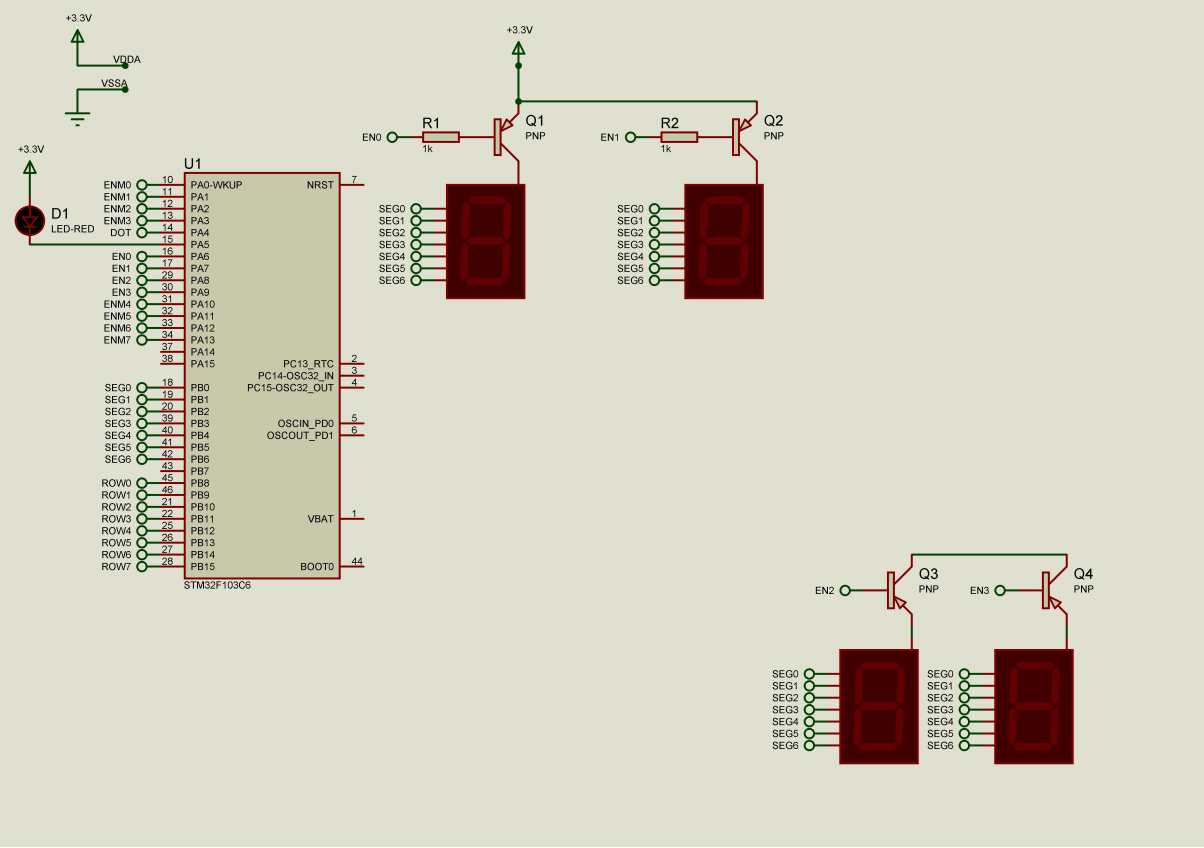
Như vậy hàm `HAL_TIM_PeriodElapsedCallback` được gọi mỗi 10ms. Một khoảng thời gian t tương ứng với $N = t/10\,\text{ms}$ lần ngắt.

1.2 Phân bổ chân

Nhãn	**Chân**	**Chức năng**
LED_RED	PA5	LED đỏ (active low)
DOT	PA4	2 LED dấu hai chấm (active low)
EN0 – EN3	PA6 – PA9	Bật 7SEG qua PNP (mức 0 = bật)
SEG0 – SEG6	PB0 – PB6	Đoạn a – g (mức 0 = sáng)
ENM0 – ENM7	PA2, PA3, PA10 – PA15	Ngõ vào ULN2803
ROW0 – ROW7	PB8 – PB15	Dữ liệu LED matrix

2 Exercise 1

2.1 Report 1: Schematic



Hình 1: Schematic Exercise 1 – Proteus project (GitHub)

2.2 Report 2: Source code

Các hàm hỗ trợ cho 7SEG (dùng lại `display7SEG` từ Lab 1):

```
uint8_t segTable[10][7] = {
    // a  b  c  d  e  f  g      (0 = segment ON, common anode)
    {  0, 0, 0, 0, 0, 0, 1 },   // 0
    {  1, 0, 0, 1, 1, 1, 1 },   // 1
    {  0, 0, 1, 0, 0, 1, 0 },   // 2
    {  0, 0, 0, 0, 1, 1, 0 },   // 3
    {  1, 0, 0, 1, 1, 0, 0 },   // 4
    {  0, 1, 0, 0, 1, 0, 0 },   // 5
    {  0, 1, 0, 0, 0, 0, 0 },   // 6
    {  0, 0, 0, 1, 1, 1, 1 },   // 7
    {  0, 0, 0, 0, 0, 0, 0 },   // 8
    {  0, 0, 0, 0, 1, 0, 0 }    // 9
};

void display7SEG(int num) {
```

```
    HAL_GPIO_WritePin(SEG0_GPIO_Port, SEG0_Pin, segTable[num][0]);
    HAL_GPIO_WritePin(SEG1_GPIO_Port, SEG1_Pin, segTable[num][1]);
    HAL_GPIO_WritePin(SEG2_GPIO_Port, SEG2_Pin, segTable[num][2]);
    HAL_GPIO_WritePin(SEG3_GPIO_Port, SEG3_Pin, segTable[num][3]);
    HAL_GPIO_WritePin(SEG4_GPIO_Port, SEG4_Pin, segTable[num][4]);
    HAL_GPIO_WritePin(SEG5_GPIO_Port, SEG5_Pin, segTable[num][5]);
    HAL_GPIO_WritePin(SEG6_GPIO_Port, SEG6_Pin, segTable[num][6]);
}

// Turn off all 4 digits (EN = 1 -> PNP off)
void turnOffAll7SEG(void) {
    HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin | EN1_Pin | EN2_Pin | EN3_Pin, GPIO_PIN_SET);
}

// Turn on one digit (EN = 0 -> PNP on)
void turnOn7SEG(int index) {
    switch (index) {
        case 0: HAL_GPIO_WritePin(EN0_GPIO_Port, EN0_Pin, GPIO_PIN_RESET); break;
        case 1: HAL_GPIO_WritePin(EN1_GPIO_Port, EN1_Pin, GPIO_PIN_RESET); break;
        case 2: HAL_GPIO_WritePin(EN2_GPIO_Port, EN2_Pin, GPIO_PIN_RESET); break;
        case 3: HAL_GPIO_WritePin(EN3_GPIO_Port, EN3_Pin, GPIO_PIN_RESET); break;
    }
}
```

Hàm ngắt timer:

```
int segCounter = 50;   // 50 x 10ms = 500ms
int currentSeg = 0;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance != TIM2) return;

    segCounter--;
    if (segCounter <= 0) {
        segCounter = 50;
        turnOffAll7SEG();              // turn off the old digit first
        if (currentSeg == 0) {
            display7SEG(1);            // "1" on the first digit
            turnOn7SEG(0);
```

```
            currentSeg = 1;
        } else {
            display7SEG(2);              // "2" on the second digit
            turnOn7SEG(1);
            currentSeg = 0;
        }
    }
}
```

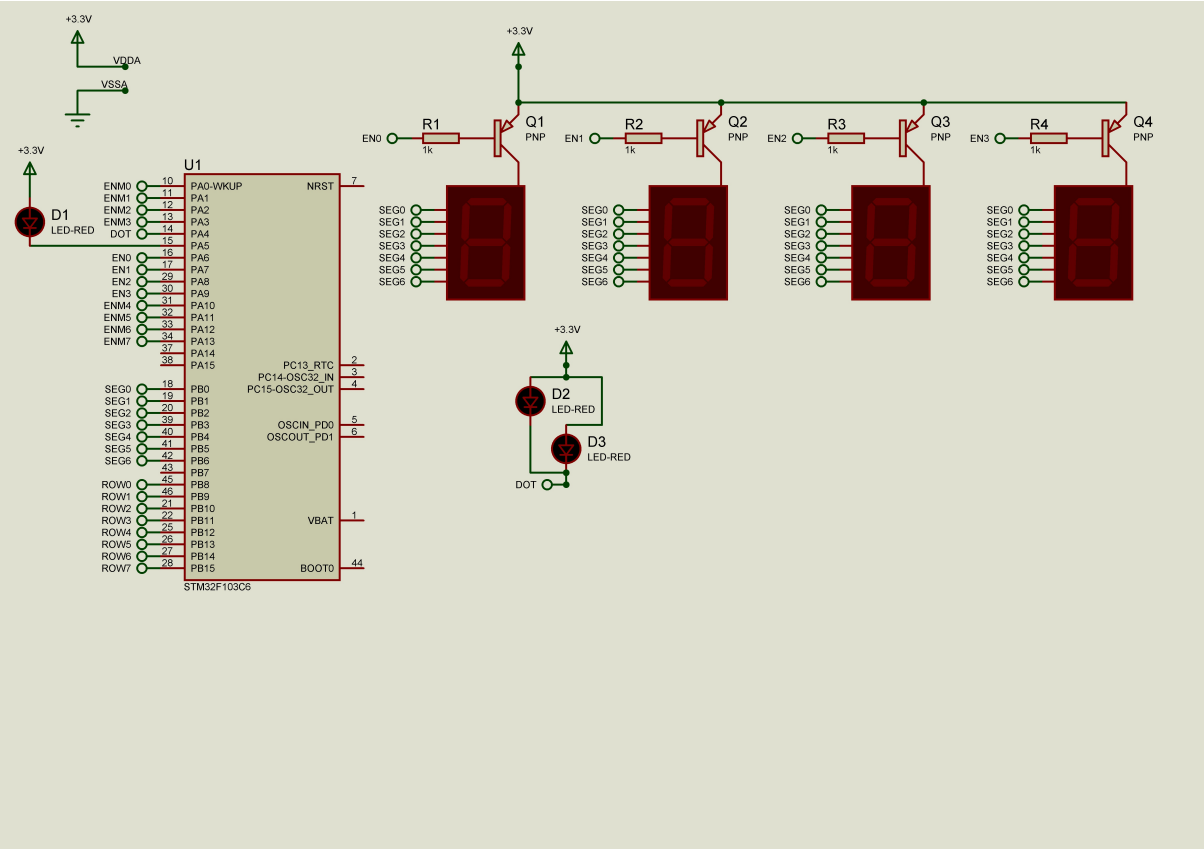
2.3 Short question

Mỗi LED 7 đoạn sáng trong $T_S = 500\,\text{ms}$ (50 lần ngắt). Với 2 LED, chu kỳ quét là $2T_S = 1\,\text{s}$, do đó tần số quét:

$$f = \frac{1}{2 \times 0.5\,\text{s}} = 1\,\text{Hz}$$

3 Exercise 2

3.1 Report 1: Schematic



Hình 2: Schematic Exercise 2 – 8 – Proteus project (GitHub)

3.2 Report 2: Source code

```
int segCounter = 50;     // 500ms per digit
int dotCounter = 100;    // toggle DOT every 1000ms
int currentSeg = 0;      // next digit: 0..3

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance != TIM2) return;

    // ---- scan 7SEG: 1 2 3 0 ----
    segCounter--;
    if (segCounter <= 0) {
        segCounter = 50;
        turnOffAll7SEG();
        switch (currentSeg) {
            case 0: display7SEG(1); turnOn7SEG(0); break;
            case 1: display7SEG(2); turnOn7SEG(1); break;
            case 2: display7SEG(3); turnOn7SEG(2); break;
            case 3: display7SEG(0); turnOn7SEG(3); break;
        }
        currentSeg++;
        if (currentSeg >= 4) currentSeg = 0;
    }

    // ---- blink DOT (PA4) ----
    dotCounter--;
    if (dotCounter <= 0) {
        dotCounter = 100;
        HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
    }
}
```

3.3 Short question

Mỗi LED sáng $T_S = 500\,\text{ms}$, 4 LED cho chu kỳ $4T_S = 2\,\text{s}$:

$$f = \frac{1}{4 \times 0.5\,\text{s}} = 0.5\,\text{Hz}$$

DOT được đảo trạng thái sau mỗi $1000\,\text{ms}/10\,\text{ms} = 100$ lần ngắt.

4 Exercise 3

4.1 Report 1: Hàm update7SEG

```
const int MAX_LED = 4;
int index_led = 0;
int led_buffer[4] = {1, 2, 3, 4};

```

```c
void update7SEG(int index) {
    turnOffAll7SEG();                    // turn off the current digit
    switch (index) {
        case 0: display7SEG(led_buffer[0]); turnOn7SEG(0); break;
        case 1: display7SEG(led_buffer[1]); turnOn7SEG(1); break;
        case 2: display7SEG(led_buffer[2]); turnOn7SEG(2); break;
        case 3: display7SEG(led_buffer[3]); turnOn7SEG(3); break;
        default: break;
    }
}
```

4.2 Report 2: Hàm ngắt timer

```c
int segCounter = 50;    // 500ms per digit
int dotCounter = 100;   // 1s toggle DOT

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance != TIM2) return;

    segCounter--;
    if (segCounter <= 0) {
        segCounter = 50;
        update7SEG(index_led++);        // show digit index_led, then move on
        if (index_led >= MAX_LED) index_led = 0;
    }

    dotCounter--;
    if (dotCounter <= 0) {
        dotCounter = 100;
        HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
    }
}
```

Biến `index_led` luôn nằm trong khoảng 0 – 3. Khi đổi `led_buffer` thành {5, 6, 7, 8}, 4 LED hiển thị lần lượt 5, 6, 7, 8.

5 Exercise 4

5.1 Report 1: Hàm ngắt timer

Để 4 LED được quét với tần số 1Hz: $4T_S = 1\,\text{s} \Rightarrow T_S = 250\,\text{ms} = 25$ lần ngắt.

```c
int segCounter = 25;    // 250ms per digit -> 4 digits = 1s -> 1 Hz
int dotCounter = 100;   // 1s toggle DOT

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance != TIM2) return;
```

```
6
7       segCounter--;
8       if (segCounter <= 0) {
9           segCounter = 25;
10          update7SEG(index_led++);
11          if (index_led >= MAX_LED) index_led = 0;
12      }
13
14      dotCounter--;
15      if (dotCounter <= 0) {
16          dotCounter = 100;
17          HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
18      }
19  }
```

6 Exercise 5

6.1 Report 1: Hàm updateClockBuffer

```
1  int hour = 15, minute = 8, second = 50;
2
3  void updateClockBuffer(void) {
4      led_buffer[0] = hour / 10;      // 15 -> 1
5      led_buffer[1] = hour % 10;      // 15 -> 5
6      led_buffer[2] = minute / 10;    //  8 -> 0 (leading zero)
7      led_buffer[3] = minute % 10;    //  8 -> 8
8  }
```

Phép chia nguyên cho 10 tự động tạo chữ số 0 phía trước khi giờ hoặc phút chỉ có 1 chữ số. Vòng lặp trong `main`:

```
1  updateClockBuffer();
2  while (1) {
3      second++;
4      if (second >= 60) { second = 0; minute++; }
5      if (minute >= 60) { minute = 0; hour++; }
6      if (hour >= 24)   { hour = 0; }
7      updateClockBuffer();
8      HAL_Delay(1000);
9  }
```

7 Exercise 6

Software timer được khai báo như sau và `timer_run()` được gọi trong hàm ngắt:

```
1  int timer0_counter = 0;
2  int timer0_flag = 0;
3  int TIMER_CYCLE = 10;          // timer interrupt period: 10ms
4
```

```c
void setTimer0(int duration) {
    timer0_counter = duration / TIMER_CYCLE;
    timer0_flag = 0;
}

void timer_run(void) {
    if (timer0_counter > 0) {
        timer0_counter--;
        if (timer0_counter == 0) timer0_flag = 1;
    }
}
```

Chương trình kiểm tra trong `main`:

```c
setTimer0(1000);                    // line 1
while (1) {
    if (timer0_flag == 1) {
        HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
        setTimer0(2000);
    }
}
```

Với code gốc, LED đỏ đổi trạng thái lần đầu sau 1s, sau đó cứ mỗi 2s đổi một lần.

7.1 Report 1: Bỏ dòng 1

LED đỏ **không bao giờ nhấp nháy**. Biến `timer0_counter` được khởi tạo bằng 0 nên điều kiện `timer0_counter > 0` trong `timer_run()` luôn sai. Vì vậy `timer0_flag` không bao giờ được bật lên 1, và lệnh `setTimer0(2000)` nằm trong khối `if` cũng không bao giờ được thực thi. Software timer phải được khởi động bằng `setTimer0` ít nhất một lần.

7.2 Report 2: Dòng 1 đổi thành setTimer0(1)

LED đỏ **vẫn không nhấp nháy**. Do phép chia nguyên $1/10 = 0$ nên `timer0_counter = 0`, giống hệt trường hợp bỏ dòng 1. Software timer không thể đo khoảng thời gian nhỏ hơn chu kỳ ngắt (10ms).

7.3 Report 3: Dòng 1 đổi thành setTimer0(10)

LED đỏ **nhấp nháy bình thường**. Lúc này $10/10 = 1$ nên `timer0_counter = 1 > 0`. Sau đúng 1 lần ngắt (khoảng 10ms) cờ `timer0_flag` được bật và LED đổi trạng thái gần như ngay lập tức; các lần sau cách nhau 2s nhờ `setTimer0(2000)`. Khác với hai trường hợp trên, bộ đếm có giá trị lớn hơn 0 nên timer thật sự đếm; khác với code gốc, lần đảo đầu tiên xảy ra sau 10ms thay vì 1s.

8 Exercise 7

8.1 Report 1: Vòng lặp while trong main

`HAL_Delay` được loại bỏ, đồng hồ và DOT được cập nhật mỗi 1s bằng `timer0` (1000/10 = 100 lần ngắt). Hàm ngắt chỉ còn `timer_run()` và quét 7SEG.

```
updateClockBuffer();
setTimer0(1000);
while (1)
{
    if (timer0_flag == 1) {           // every 1 second
        setTimer0(1000);
        HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);    // DOT
           moved to main
        second++;
        if (second >= 60) { second = 0; minute++; }
        if (minute >= 60) { minute = 0; hour++; }
        if (hour >= 24)   { hour = 0; }
        updateClockBuffer();
    }
}
```

9 Exercise 8

9.1 Report 1: Hàm main

Hàm `update7SEG()` được chuyển ra `main` bằng software timer thứ hai (`timer1`, 250/10 = 25 lần ngắt). Hàm ngắt chỉ còn xử lý software timer.

```
int timer1_counter = 0;
int timer1_flag = 0;

void setTimer1(int duration) {
    timer1_counter = duration / TIMER_CYCLE;
    timer1_flag = 0;
}

void timer_run(void) {
    if (timer0_counter > 0) {
        timer0_counter--;
        if (timer0_counter == 0) timer0_flag = 1;
    }
    if (timer1_counter > 0) {
        timer1_counter--;
        if (timer1_counter == 0) timer1_flag = 1;
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
```

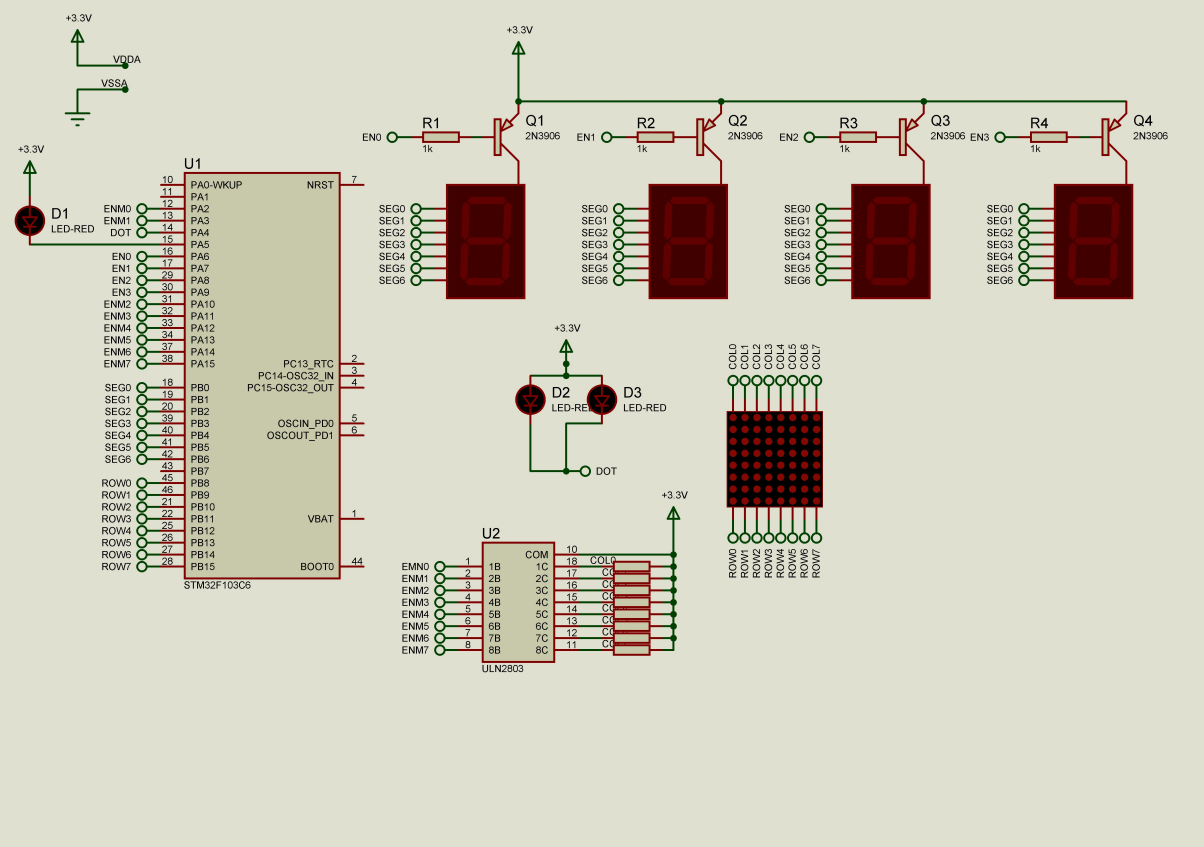
```
    if (htim->Instance == TIM2) {
        timer_run();
    }
}
```

```
updateClockBuffer();
setTimer0(1000);      // clock + DOT
setTimer1(250);       // 7SEG scanning
while (1)
{
    if (timer0_flag == 1) {           // every 1s
        setTimer0(1000);
        HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
        second++;
        if (second >= 60) { second = 0; minute++; }
        if (minute >= 60) { minute = 0; hour++; }
        if (hour >= 24)   { hour = 0; }
        updateClockBuffer();
    }

    if (timer1_flag == 1) {           // every 250ms
        setTimer1(250);
        update7SEG(index_led++);
        if (index_led >= MAX_LED) index_led = 0;
    }
}
```

10 Exercise 9

10.1 Report 1: Schematic



Hình 3: Schematic Exercise 9 – 10 – Proteus project (GitHub)

Trong mạch này, ma trận được đặt sao cho các chân ROW0 – ROW7 (PB8 – PB15) nối vào phía anode và điều khiển 8 cột dọc, còn ULN2803 (qua ENM0 – ENM7) kéo phía cathode xuống 0V để chọn từng hàng ngang. Do đó ma trận được quét theo từng hàng: một hàng sáng khi ENM tương ứng bằng 1, một điểm sáng khi ROW tương ứng bằng 1.

10.2 Report 2: Hàm updateLEDMatrix

```
const int MAX_LED_MATRIX = 8;
int index_led_matrix = 0;

// Each byte = one horizontal row (selected by ENMr), bit c =
   column c (ROWc). 1 = ON
uint8_t matrix_buffer[8] = {
    0b00011000,    // . . . # # . . .
    0b00111100,    // . . # # # # . .
    0b01100110,    // . # # . . # # .
    0b11000011,    // # # . . . . # #
```

```
    0b11111111,   // # # # # # # # #
    0b11111111,   // # # # # # # # #
    0b11000011,   // # # . . . . # #
    0b11000011    // # # . . . . # #
};

// Turn off all rows: ENM = 0 -> ULN2803 releases all lines
void clearAllColumns(void) {
    HAL_GPIO_WritePin(ENM0_GPIO_Port,
        ENM0_Pin | ENM1_Pin | ENM2_Pin | ENM3_Pin |
        ENM4_Pin | ENM5_Pin | ENM6_Pin | ENM7_Pin, GPIO_PIN_RESET
           );
}

// Show row "index": write 8 data bits to ROW, then enable that
   row (ENM = 1)
void updateLEDMatrix(int index) {
    clearAllColumns();

    uint8_t pattern = matrix_buffer[index];
    for (int i = 0; i < 8; i++) {
        // ROW0..ROW7 = PB8..PB15 -> ROW0_Pin << i; bit = 1 ->
           SET (ON)
        HAL_GPIO_WritePin(ROW0_GPIO_Port, ROW0_Pin << i,
                          (pattern & (1 << i)) ? GPIO_PIN_SET :
                             GPIO_PIN_RESET);
    }

    switch (index) {
        case 0: HAL_GPIO_WritePin(ENM0_GPIO_Port, ENM0_Pin,
           GPIO_PIN_SET); break;
        case 1: HAL_GPIO_WritePin(ENM1_GPIO_Port, ENM1_Pin,
           GPIO_PIN_SET); break;
        case 2: HAL_GPIO_WritePin(ENM2_GPIO_Port, ENM2_Pin,
           GPIO_PIN_SET); break;
        case 3: HAL_GPIO_WritePin(ENM3_GPIO_Port, ENM3_Pin,
           GPIO_PIN_SET); break;
        case 4: HAL_GPIO_WritePin(ENM4_GPIO_Port, ENM4_Pin,
           GPIO_PIN_SET); break;
        case 5: HAL_GPIO_WritePin(ENM5_GPIO_Port, ENM5_Pin,
           GPIO_PIN_SET); break;
        case 6: HAL_GPIO_WritePin(ENM6_GPIO_Port, ENM6_Pin,
           GPIO_PIN_SET); break;
        case 7: HAL_GPIO_WritePin(ENM7_GPIO_Port, ENM7_Pin,
           GPIO_PIN_SET); break;
        default: break;
    }
}
```

Hàm được gọi trong `main` bằng software timer thứ ba, mỗi hàng 10ms, tức 8 hàng mất 80ms (12.5Hz):

```
setTimer2(10);        // LED matrix scanning
while (1) {
    /* ... timer0 and timer1 as in Exercise 8 ... */
    if (timer2_flag == 1) {           // every 10ms
        setTimer2(10);
        updateLEDMatrix(index_led_matrix++);
        if (index_led_matrix >= MAX_LED_MATRIX) index_led_matrix
           = 0;
    }
}
```

11 Exercise 10

11.1 Report 1: Mô tả giải pháp và source code

Animation được chọn là dòng chữ **TRINH** chạy từ phải sang trái. Giải pháp gồm 3 phần:

- **Dữ liệu chữ** `text_cols[]`: font 5x7, mỗi byte là một cột của chữ (bit r = hàng r), các chữ cách nhau 1 cột trống, đầu chuỗi có 8 cột trống để chữ chạy vào từ mép phải. Tổng cộng 38 cột.
- **loadFrame()**: lấy 8 cột liên tiếp bắt đầu từ `text_offset`, chuyển sang dạng hàng và ghi vào `matrix_buffer`. Hàm `updateLEDMatrix` của Exercise 9 được giữ nguyên để quét.
- **shiftTextLeft()**: tăng `text_offset` thêm 1 (quay vòng) và tạo lại khung hình, gọi bởi software timer thứ tư (`timer3`) mỗi 200ms.

Việc quét (timer2, 10ms) và việc dịch chữ (timer3, 200ms) hoạt động độc lập nên hình luôn liền mạch. Một vòng chạy hết chuỗi mất $38 \times 0.2\,\text{s} = 7.6\,\text{s}$.

```
// 5x7 font: each byte = one column of the text, bit r = row r (
   bit0 = top)
const uint8_t text_cols[] = {
    0,0,0,0,0,0,0,0,                          // 8 blank columns
    0x01,0x01,0x7F,0x01,0x01, 0,              // T
    0x7F,0x09,0x19,0x29,0x46, 0,              // R
    0x00,0x41,0x7F,0x41,0x00, 0,              // I
    0x7F,0x04,0x08,0x10,0x7F, 0,              // N
    0x7F,0x08,0x08,0x08,0x7F, 0               // H
};
#define TEXT_LEN (sizeof(text_cols))          // 38 columns
int text_offset = 0;                          // text column at
   the left edge

// Take 8 columns starting at text_offset and convert them to
   rows
void loadFrame(void) {
    for (int r = 0; r < 8; r++) {
```

```
        uint8_t rowBits = 0;
        for (int c = 0; c < 8; c++) {
            uint8_t col = text_cols[(text_offset + c) % TEXT_LEN
               ];
            if (col & (1 << r)) rowBits |= (1 << c);
        }
        matrix_buffer[r] = rowBits;
    }
}

// Shift the text one column to the left (wrap around)
void shiftTextLeft(void) {
    text_offset = (text_offset + 1) % TEXT_LEN;
    loadFrame();
}
```

```
updateClockBuffer();
setTimer0(1000);      // clock + DOT
setTimer1(250);       // 7SEG scanning
setTimer2(10);        // LED matrix scanning
loadFrame();          // first frame
setTimer3(200);       // shift text every 200ms
while (1)
{
    if (timer0_flag == 1) {                 // every 1s
        setTimer0(1000);
        HAL_GPIO_TogglePin(DOT_GPIO_Port, DOT_Pin);
        second++;
        if (second >= 60) { second = 0; minute++; }
        if (minute >= 60) { minute = 0; hour++; }
        if (hour >= 24)   { hour = 0; }
        updateClockBuffer();
    }

    if (timer1_flag == 1) {                 // every 250ms
        setTimer1(250);
        update7SEG(index_led++);
        if (index_led >= MAX_LED) index_led = 0;
    }

    if (timer2_flag == 1) {                 // every 10ms
        setTimer2(10);
        updateLEDMatrix(index_led_matrix++);
        if (index_led_matrix >= MAX_LED_MATRIX) index_led_matrix
           = 0;
    }

    if (timer3_flag == 1) {                 // every 200ms
        setTimer3(200);
        shiftTextLeft();
    }
```

```
35 }
```